

Πανεπιστήμιο Κρήτης - Τμήμα Επιστήμης Υπολογιστών

HY252 - Αντικειμενοστραφής Προγραμματισμός Java

Διδάσκων: Ι. Τζίτζικας - Χειμερινο Εξάμηνο 2025-2026

[PROJECT PHASE #1]

Πετράκης Ραφαήλ Ιωάννης

csd5748

15/11/25 – 01/12/25

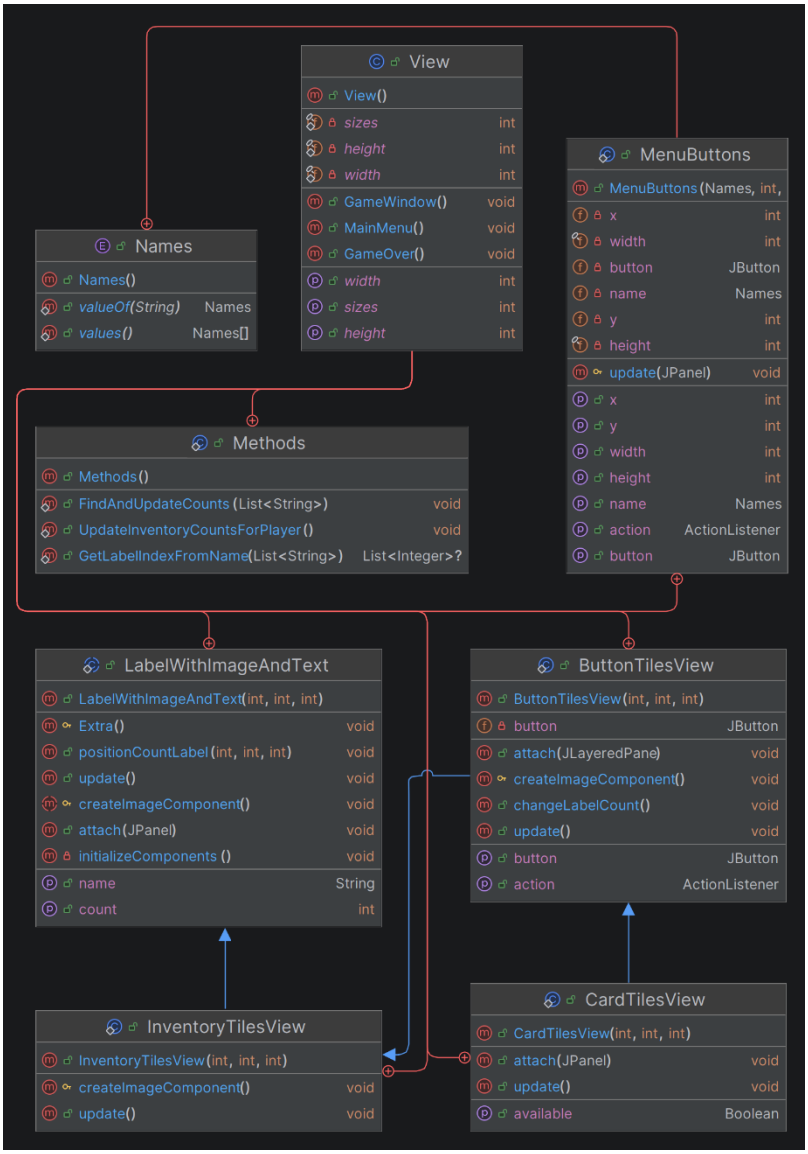
- ## Περιεχόμενα:

1. **Εισαγωγή** **2**
2. **Περιγραφή του Πακέτου View** **2-3**
3. **Περιγραφή του Πακέτου Model** **4-5**
4. **Περιγραφή του Πακέτου Controller** **6**
5. **Αλληλεπιδράσεις και διαγράμματα UML** **7**
6. **Λειτουργικότητα Κώδικα** **8**
7. **Συμπεράσματα και Σχόλια για την Φάση #1** **8**

1. Εισαγωγή:

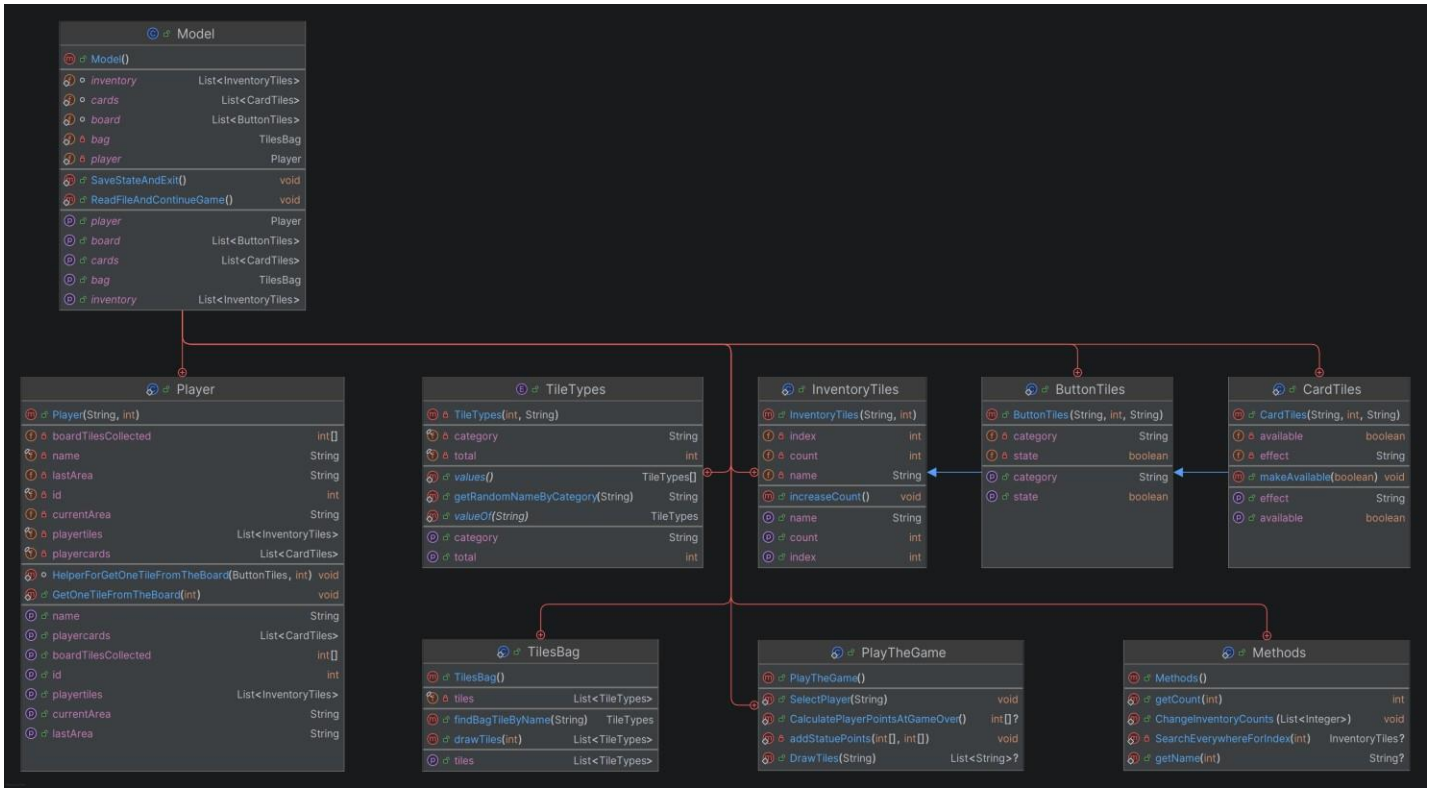
Το συγκεκριμένο Project αποτελεί την υλοποίησης του επιτραπέζιου παιχνιδιού «Amphipolis», είναι βασισμένο στο μοντέλο αρχιτεκτονικής «MVC» (Model-View-Controller) και θα δημιουργηθεί χρησιμοποιώντας Java στο IntelliJ. Έτσι, ο στόχος είναι η κλάση Controller να λειτουργεί ως ο εγκέφαλος του παιχνιδιού, συνδέοντας τα δεδομένα του παιχνιδιού, που βρίσκονται στο πακέτο Model, με την γραφική διεπαφή του, η οποία βρίσκεται στο πακέτο View. Στην συνέχεια της αναφοράς λοιπόν, θα περιγράφουνε τα (πρότυπα) πακέτα Model, View & Controller …

2. Περιγραφή του Πακέτου View:



Αρχικά, έχουμε UML διάγραμμα που δημιούργησε το IntelliJ (Ultimate IDE) παραπάνω. Σημείωση: έχω αφαιρέσει τις κλάσεις τις οποίες δεν δημιούργησα εγώ (πχ. JFrame, κτλ.). Λοιπόν, ο λόγος που ξεκινάμε από το View.java θα γίνει κατανοητός αργότερα. Για τώρα, έχουμε αρχικά την κλάση View που περιέχει τον constructor, ένα προκαθορισμένο height, width, size, καθώς και getters για αυτά. Επίσης, έχουμε τις μεθόδους: MainMenu που είναι το αρχικό παράθυρο που ανοίγει κατά την εκκίνηση και θα μπορούμε να διαλέξουμε τι mode θα παίξουμε (4Players, 1Player ή Load&Continue), GameWindow που θα εμφανίζει το κύριο παράθυρο παιχνιδιού, και GameOver όπου θα εμφανίζει ένα άλλο παράθυρο με τον νικητή του παιχνιδιού (και τους συνολικούς του πόντους). Με λίγα λόγια, το κάθε παράθυρο θα εμφανίζεται με εντολή του Controller (μέσω κάποιου κουμπιού) και θα χρησιμοποιούν δεδομένα από το Model. Έπειτα, η κλάση MenuButtons που χρησιμοποιεί ένα Enumeration Names απλά θα δημιουργεί τα απαραίτητα κουμπιά που θα χρησιμοποιηθούν γενικά στην εφαρμογή (4Players, 1Player, Load&Continue, Draw Tiles, End Turn, SaveState&Exit, κτλ.). Αυτά τα κουμπιά θα έχουν ένα name (από το enum Names), height, width, συντεταγμένες x,y, καθώς και setters/getters για όλα αυτά. Με λίγα λόγια, το κάθε κουμπί θα αποθηκεύεται στο View αλλά θα χρησιμοποιείται στον Controller (μέσω ActionListeners). Μετά, έχουμε μια κλάση Methods που θα περιέχει διάφορες μεθόδους που μπορεί να χρειαστούμε. Ο λόγος που οι μέθοδοι θα είναι στην κλάση αυτή είναι για λόγους ευκολίας εύρεσης τους (απλά γράφουμε "View.Methods.") αλλά και επεξεργασίας τους. Τώρα, για τα κύρια στοιχεία: Tiles, Cards, κτλ., η ιδέα είναι η εξής: αφού στο View δεν μπορούμε να αποθηκεύσουμε όλα τα δεδομένα των πλακιδίων, αλλά μόνο όσα αφορούν το GUI, θα δημιουργήσουμε μια Abstract Class LabelWithImageAndText, η οποία θα προσθέτει στο παράθυρο δύο ετικέτες JLabels: ένα Image και ένα Text. Έτσι, χρειάζεται δύο συντεταγμένες x,y, μεγέθη height, width, ένα index*, καθώς και setters/getters. Ακόμη, μεθόδους createImageComponents, initializeComponents, positionCountLabels, attach και update που απλά τις δημιουργούν, αρχικοποιούν, τοποθετούν κατάλληλα, προσθέτουν και ενημερώνουν μελλοντικά στο παράθυρο αντίστοιχα. Με λίγα λόγια, έτσι έχουμε μία εικόνα (κάποιου πλακιδίου) και ένα κείμενο (το πλήθος του) με ένα μοναδικό αριθμό index πάνω στο παράθυρο. Εύκολα, λοιπόν, μπορεί αυτό να φτιαχτεί η Class InventoryTilesView extends LabelWithImageAndText και να αντιπροσωπεύει τα πλακίδια που μαζεύει ο κάθε παίκτης (inventory). Επίσης, εύκολα μπορούμε να φτιάξουμε μια Class ButtonTilesView extends InventoryTilesView που θα αντιπροσωπεύει τα πλακίδια που θα υπάρχουν στο ταμπλό (board tiles), με την μοναδική διαφορά να είναι πως η εικόνα θα είναι πάνω σε ένα κουμπί (και παραμένει το κείμενο). Τέλος, θα φτιάξουμε και την Class CardTilesView extends LabelWithImageAndText που είναι το ίδιο με την προηγούμενη (στο View) αλλά αντιπροσωπεύει τις κάρτες που έχει ο κάθε παίκτης. Σημείωση: για τις κάρτες, ξέρουμε πως το πλήθος τους είναι σταθερό και ίσο με 1. Ωστόσο, θα μπορούσαμε να χρησιμοποιήσουμε την διαθέσιμη ετικέτα (JLabel Text) για να δείχνουμε εάν η κάρτα είναι ενεργή (π.χ.). Τελικά, τώρα έχουμε (υποθετικά) ότι χρειαζόμαστε για το GUI του παιχνιδιού – εννοείται θα προσθέσουμε σε αντίστοιχη μέθοδο την εικόνα του ταμπλό (JLaredPane background), κείμενο για το ποιος παίζει (turn) αλλά και χρόνο (timer), και άλλες σχετικές πληροφορίες … Αυτά για το View λοιπόν, μπορούμε να συνεχίσουμε με το Model (όπου σχετίζεται άμεσα και το συμπληρώνει αλλά δεν επικοινωνεί άμεσα μαζί του) …

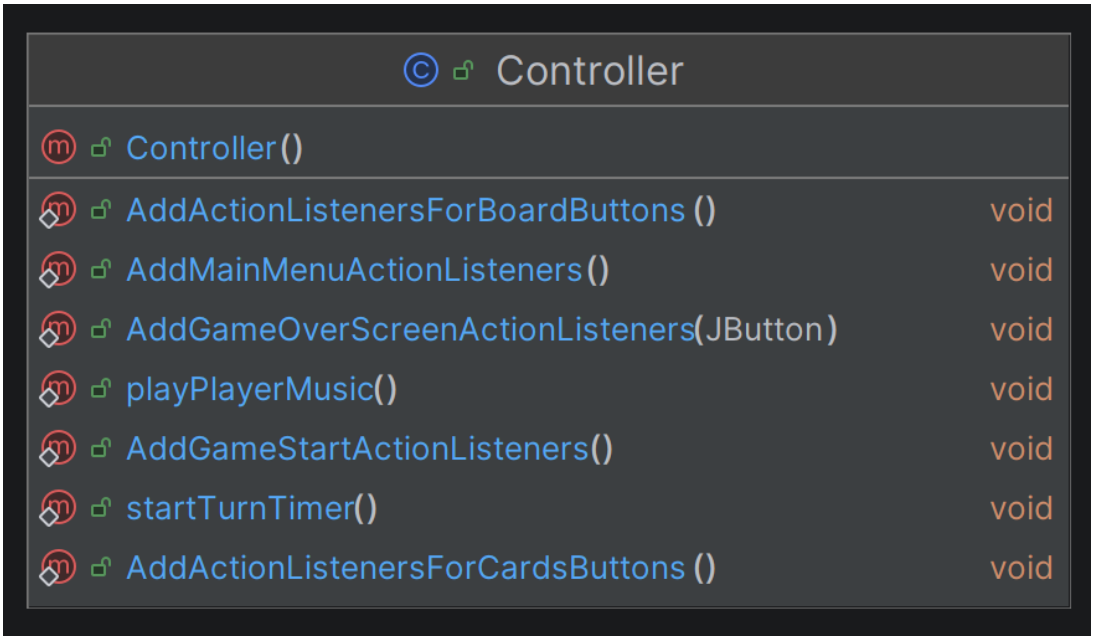
3. Περιγραφή του Πακέτου Model:



Αρχικά, έχουμε UML διάγραμμα που δημιούργησε το IntelliJ (Ultimate IDE) παραπάνω. Λοιπόν, όπως είπαμε, εδώ αποθηκεύουμε όλα τα δεδομένα, ενώ εφαρμόζουμε λογικές πράξεις πάνω τους. Έτσι, αφού έχουμε αποφασίσει πως θα είναι το GUI, η ιδέα της συγκεκριμένης οργάνωσης είναι η εξής: θα έχουμε παρόμοιες κλάσεις, όμως εδώ θα αποθηκεύσουμε δεδομένα που δεν σχετίζονται καθόλου με το GUI, αλλά το αφορούν … Οπότε, η κύρια κλάση Model θα περιέχει αρκετά προκαθορισμένα πράγματα, όπως: player (ο παίκτης που παίζει αυτή την στιγμή), bag (η κοινή για όλους σακούλα με όλα τα πλακίδια), inventory (όλα τα πλακίδια που θα μπορεί αποκτήσει κάθε παίκτης), board (όλα τα πλακίδια που θα υπάρχουν στο ταμπλό), cards (οι κάρτες που θα έχει κάθε παίκτης), μεθόδους που θα επεξεργάζονται όλα τα προηγούμενα (κυρίως κατά την αποθήκευση παιχνιδιού και συνέχισή του), καθώς και setters/getters για όλα αυτά. Πιο συγκεκριμένα, έχουμε class InventoryTiles που είναι ίδια με του View, απλά εδώ θα αποθηκεύουμε το όνομα (name), το απόθεμα (count), ξανά ένα δείκτη (index), διάφερες βοηθητικές μεθόδους και setters/getters. Σημείωση: όλοι οι δείκτες (indexes) ανάμεσα στο Model και View είναι ακριβώς ίδιοι, ώστε για έναν τυχαίο, ο Controller να μπορεί να αντλήσει όλες τις πληροφορίες από τα Model και View ταυτόχρονα. Επιπλέον, έχουμε class ButtonTiles extends InventoryTiles, αφού χρειαζόμαστε και για τα κουμπιά του ταμπλό όνομα, πλήθος, δείκτη, ενώ προσθέτουμε κατάσταση (state) που θ είναι on ή off, κατηγορία (category) που βρίσκεται στο ταμπλό (πάνω-αριστερά, πάνω-δεξιά, κάτω-δεξιά, κάτω-αριστερά), και …

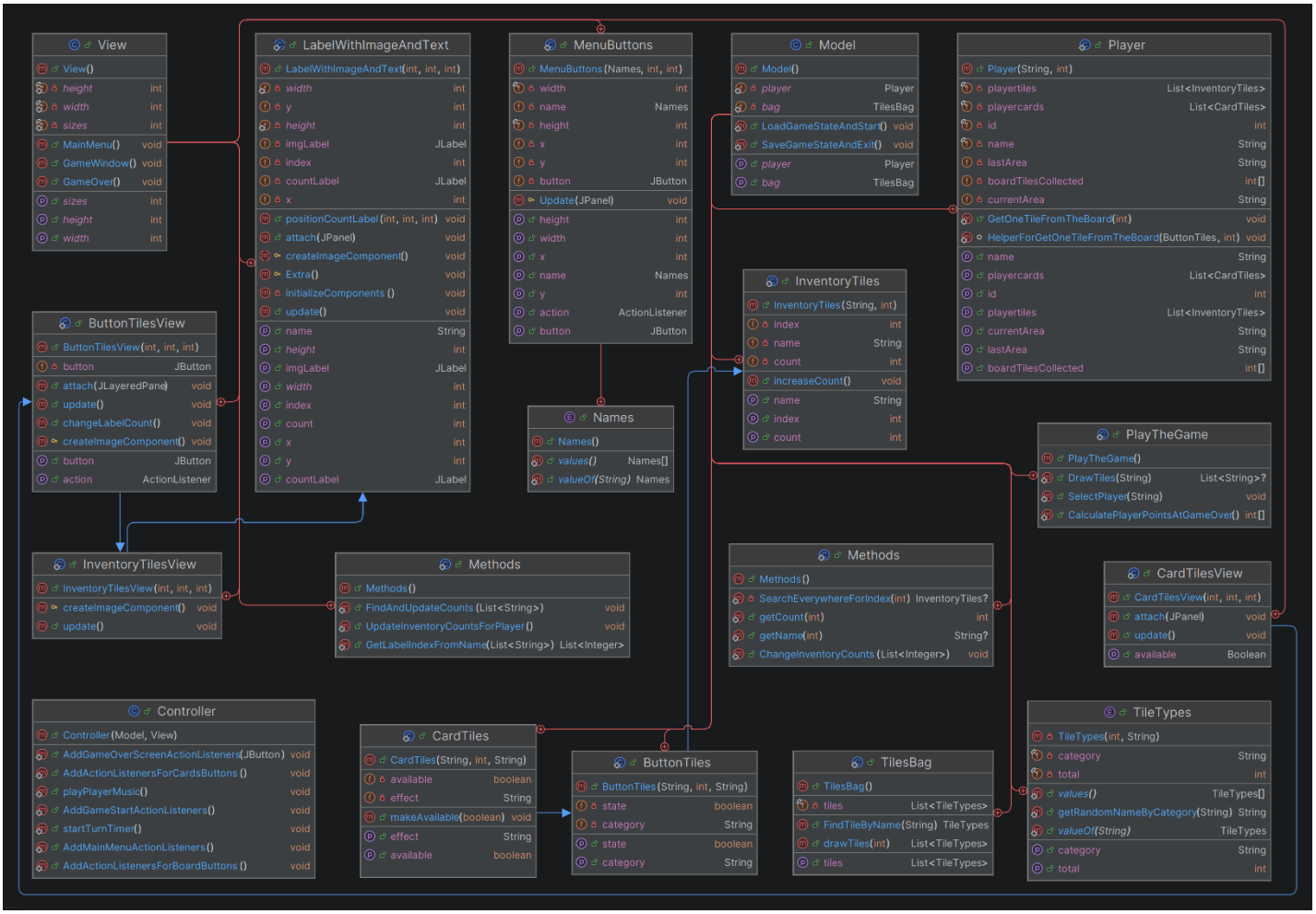
… νέα setters/getters. Ακόμη, class CardTiles extends ButtonTiles, αφού προφανώς και οι κάρτες είναι κουμπιά με τα ίδια χαρακτηρίστηκα, αλλά έχουν και μια δύναμη (effect/power – το τι κάνει η κάθε κάρτα), διαθεσιμότητα (available – αν έχει χρησιμοποιηθεί η μία διαθέσιμη χρήση), και νέα setters/getters. Εκτός από αυτά, τώρα, εδώ θα δημιουργήσουμε μια enum TileTypes όπου θα μπορούμε με εύκολο τρόπο να αρχικοποιήσουμε τα πλακίδια της σακούλας μας. Αυτά θα έχουν όνομα (name), πλήθος (total), κατηγορία (category), getters, και έναν constructor. Σημείωση: αυτή η δομή δεδομένων είναι απαραίτητη για να μας διευκολύνει στην προσθήκη νέων πλακιδίων ή επεξεργασία των τωρινών στο μέλλον. Έτσι, στην class TilesBag όπου θα αντιπροσωπεύει την σακούλα μας, περιέχοντας όλα τα πλακίδια στην αρχή του παιχνιδιού, μπορούμε με μια απλή μέθοδο να εισάγουμε τα πλακίδια σε μια δυναμική λίστα (ArrayList<>), καθώς και να αφαιρούμε με μία άλλη (ένα – ένα). Να μην ξεχάσουμε τον constructor, getter, και finder που θα έχουμε. Έπειτα, θα χρειαστούμε σίγουρα μια class Player, όπου θα αντιπροσωπεύει τους παίκτες μας, με ένα όνομα (name), αριθμό (id), πλακίδια (inventory μέσω αντίστοιχης κλάσης), κάρτες (ομοίως), τα πλακίδια μου μάζεψε τον τωρινό γύρο (int[] boardTilesCollected – για να ελέγχουμε πόσα και από που (κατηγορία) μπορεί να επιλέξει), τωρινή περιοχή (currentArea – η κατηγορία του πλακιδίου που διάλεξε από το ταμπλό (λόγω των καρτών)), προηγούμενη περιοχή (lastArea – η τελευταία κατηγορία (λόγω της κάρτας Coder)), setters/getters (για όλα), constructor (για αρχικοποίηση κυρίως του inventory και cards μέσω των αντίστοιχων κλάσεων), και τέλος μία (μεγάλη) μέθοδο που θα πρέπει να ελέγχει εάν μπορεί να επιλέξει ένα συγκεκριμένο πλακίδιο από το ταμπλό. Επόμενη είναι η class PlayTheGame, η οποία είναι υπεύθυνη για την λειτουργικότητα του παιχνιδιού. Θα έχει constructor, μια μέθοδο SelectPlayer που επιλέγει είτε έναν τυχαίο παίχτη (αρχή παιχνιδιού), είτε τον επόμενο (τήρηση σειράς) ανάλογα, μια μέθοδο DrawTiles που είτε τραβάει αυτόματα 4 τυχαία πλακίδια από την σακούλα (αρχή παιχνιδιού), είτε ο κάθε παίκτης τραβάει τα δικά του (μέσω κατάλληλου κουμπιού) και θα τα τοποθετεί στο ταμπλό, καθώς και μια (τεράστια) μέθοδο CalculatePlayerPointsAtGameOver που απλά θα υπολογίζει όλους τους πόντους (με περιπτώσεις) του κάθε παίκτη στο τέλος του παιχνιδιού. Σημείωση: όλοι οι πόντοι του κάθε πλακιδίου θα ορίζονται τοπικά στην συγκεκριμένη μέθοδο - δεν θα μπορούσαν να οριστούν ως πεδίο της αρχικής κλάσης (InventoryTiles), αφού κληρονομείται από κουμπιά, αλλά εκτός από αυτό, δεν νομίζω πως υπάρχει λόγος – μια άλλη καλή ιδέα θα ήταν η αρχικοποίηση σε μια λίστα ως πεδίο του Model, όπως συμβαίνει με τα πλακίδια. Τέλος, έχουμε και εδώ μια class Methods, η οποία θα περιέχει διάφορες μεθόδους που μπορεί να χρειαστούν ή να φανούν χρήσιμοι (όπως εύρεση ενός index για παράδειγμα), για παρόμοιους λόγους. Οπότε, το συγκεκριμένο Model, ενώ έχει κάποια διαφορετικά πραγματάκια, είναι παρόμοιο με το View, αφού θα πρέπει να «σπάσουμε» στα δύο την αποθήκευση του κάθε στοιχείου (πλακιδίου) κατάλληλα με το μοντέλο MVC … Και, τώρα θεωρητικά έχουμε ένα παράθυρο που περιέχει τα πάντα, αλλά δεν μπορούμε να αλληλοεπιδράσουμε μαζί του. Ας περάσουμε για τον λόγο αυτό λοιπόν στον Controller …

4. Περιγραφή του Πακέτου Controller:



Αρχικά, έχουμε UML διάγραμμα που δημιούργησε το IntelliJ (Ultimate IDE) παραπάνω. Λοιπόν, το αρχείο του Controller θα είναι αρκετά απλό, αφού και ο στόχος του είναι συγκεκριμένος: θα ελέγχει όλα τα κουμπιά μεταξύ των διάφορων παραθύρων του View (ActionListeners), ενώ θα ζητάει/παίρνει διάφορα δεδομένα από το Model (μέσω getters) και θα τα δίνει στο View για να τα εμφανίσει στον χρήστη. Σημείωση: σύμφωνα με το μοντέλο MVC, ο Controller θα είναι ο συνδετικός κρίκος μεταξύ των Model – View … Οπότε, θα έχουμε την βασική κλάση Controller που θα περιέχει ένα constructor και μερικές μεθόδους. Αρχικά, όπως είπαμε, θέλουμε μια μέθοδο που θα δημιουργεί για κάθε κουμπί (λογικά 8 συνολικά) κάθε παραθύρου (λογικά 3 συνολικά) έναν ActionListener, ο οποίος θα εκτελεί αντίστοιχες εντολές. Έτσι, θα έχουμε AddMainMenuActionListeners(), AddGameStartActionListeners(), AddGameOverScreenActionListeners(). Επίσης, αφού πριν είπαμε πως θα χωρίσουμε στο κύριο παράθυρο διαφορετικές (αλλά σχεδόν ίδιες) κλάσεις για τα κουμπιά των πλακιδίων του ταμπλό και των καρτών, θα χρειαστούμε AddActionListenersForBoardButtons() και AddActionListenersForCardsButtons(). Τέλος, το παιχνίδι προϋποθέτει ένα χρονόμετρο αντίστροφης μέτρησης των 30 δευτερολέπτων (timer) για τον γύρο του κάθε παίκτη, καθώς και μουσική υπόκρουση καθ' όλη την διάρκεια του κάθε γύρου (music) – διαφορετική για κάθε παίκτη. Άρα, θα έχουμε και startTurnTimer(), playPlayerMusic(). Σημείωση: αυτά τα δύο τελευταία στοιχεία θα τα ελέγχει ο Controller και όχι το View … Τελικά, έχουμε έτοιμο και τον τρόπο αλληλεπίδρασης του χρήστη με το παράθυρο του παιχνιδιού, οπότε είμαστε (υποθετικά) έτοιμοι και τελειώσαμε …

5. Αλληλεπιδράσεις και διαγράμματα UML:



Αρχικά, έχουμε UML διάγραμμα που δημιούργησε το IntelliJ (Ultimate IDE) παραπάνω. Λοιπόν, σύμφωνα με όσα ειπώθηκαν προηγουμένως, έτσι πρόκειται να είναι το παιχνίδι που θα δημιουργήσουμε (πιθανότατα). Όπως και να έχει, η λογική του μοντέλου MVC θα παραμείνει ως έχει. Για τον λόγο αυτό, όπως είπαμε ήδη, ένα πλακίδιο δεν θα μπορούσε να έχει την δική του ξεχωριστή κλάση Tile. Αυτό συμβαίνει επειδή για να εμφανίσουμε έναν συγκεκριμένο αριθμό από πλακίδια (πχ. 16 landslides) σε ένα παράθυρο και να το δει ο χρήστης, χρειαζόμαστε, εκτός των άλλων, το όνομα της εικόνας (πχ. Landslide) του και τον (μεταβλητό) αριθμό του (πχ. 16). Όμως, αυτά τα δύο πεδία δεν μπορούν να αποθηκευτούν σε κανένα πακέτο μαζί – η εικόνα αφορά το View (GUI) και ο αριθμός το Model (Data). Έτσι, χωρίσαμε τα δεδομένα του κάθε πλακιδίου (Tile), κάθε κάρτας (Card), κτλ. σε μία κλάση του Model και μία του View. Για να επικοινωνήσουν μεταξύ τους, χρησιμοποιείται ο Controller. Τα πακέτα Model & View δεν γνωρίζουν το ένα το άλλο και δεν επικοινωνούν μεταξύ τους. Επίσης, οι κονστράκτορες των πακέτων αυτών απλά αρχικοποιούν όλα τα απαραίτητα δεδομένα, δεν εκτελούν τίποτα από μόνα τους. Με λίγα λόγια, το Model δίνει και παίρνει από και στον Controller, ο Controller δίνει και παίρνει από και προς το View …

6. Λειτουργικότητα Κώδικα:

* Ο κώδικας δεν είναι ακόμα έτοιμος και το παιχνίδι δεν είναι ακόμα ολοκληρωμένο …
[Περισσότερες πληροφορίες περιέχονται στο αρχείο «ReadMe.md» σε αυτόν τον φάκελο]

7. Συμπεράσματα και Σχόλια για την Φάση #1:

Λοιπόν, εδώ θα ήθελα να αναφέρω τα σχόλιά μου σχετικά με την πρώτη φάση του πρότξεκτ. Αρχικά, προσωπικά, το συγκεκριμένο πρότζεκτ μου φάνηκε αρκετά δύσκολο. Αυτό οφείλεται σε ποικίλους λόγους, όπως το ότι είναι η πρώτη φορά που πρέπει να δημιουργήσω κάτι τόσο μεγάλο και περίπλοκο σε μία γλώσσα που πρόσφατα έμαθα, όπως το ότι είναι δύσκολο να σκεφτείς και να σχεδιάσεις μόνο τις υπογραφές χωρίς την υλοποίηση των κλάσεων και μεθόδων που χρειάζονται, όπως το ότι το μοντέλο που πρέπει να ακολουθήσουμε είναι ιδιαίτερο, ή όπως το ότι η εκφώνηση ήταν για ακόμα μια φορά απογοητευτική…..
Από την άλλη, κατάφερα να ετοιμάσω όλα τα παραδοτέα της συγκεκριμένης φάσης, τα οποία είναι τα εξής: α) «Γραπτή αναφορά» - το συγκεκριμένο αρχείο που διαβάζετε (σημείωση: είναι βασισμένο στο αρχείο template.docx). β) «UML class diagrams» - περιέχονται ατομικά αλλά και συνολικά για τα πακέτα εδώ (σημείωση: δημιουργήθηκαν μέσω του Intellij Ultimate build-in diagrams). γ) «Μοντέλο MVC» - έχει χωριστεί ο κώδικας σύμφωνα με το συγκεκριμένο μοντέλο και αυτό φαίνεται τόσο εδώ όσο και στο "/src/PhaseA/MVC/*" με τα αντίστοιχα Model.java, View.java, Controller.java αρχεία). δ) «8avadoc σχόλια» - έχουν συμπεριληφθεί όλα τα απαραίτητα σχόλια "εκ των προτέρων, εκ των υστέρων και επίσης αμετάβλητες συνθήκες" στην συγκεκριμένη μορφή, στα αντίστοιχα αρχεία που βρίσκονται στο "/src/PhaseA/CommentedMVC/*" και είναι Model.java, View.java, Controller.java (σημείωση: δημιουργήθηκαν με τέτοιο τρόπο ώστε η εντολή "javadoc -d docs src/PhaseA/CommentedMVC/*.java" να παράγει το απαιτούμενο "/docs/index.html" με 0 errors …). Επιπλέον, έχουμε: ε) «Οργάνωση των κλάσεων σε ιεραρχίες με στόχο την μέγιστη δυνατή επαναχρησιμοποίηση του κώδικα. Χρησιμοποίηση abstract κλάσεων ή/και interfaces για επίσης βασικές κλάσεις Card και Square.» - έγινε χρήση "abstract" κλάσεων, και γενικά προσπάθησα να γράψω όσο το δυνατόν λιγότερο κώδικα (επαναχρησιμοποίηση), και υπάρχει η (βασική) κλάση Card, απλά με διαφορετικό όνομα ("CardTiles"). Όμως, δεν ξέρω τι είναι η "Square" και δεν διευκρινίζεται πουθενά – μπορεί να την δημιούργησα τυχαια… στ) «Πρέπει να υπάρχουν οι μέθοδοι για τα βασικά στοιχεία του παιχνιδιού: αρχικοποίηση παιχνιδιού, τράβηγμα πλακιδίων, επιλογή πλακιδίου, χαρακτήρες, υπολογισμό πόντων, σειρά, έλεγχος νικητή …» - λοιπόν, αρχικά η αρχικοποίηση του παιχνιδιού εφαρμόζεται από πολλές και όχι μία μοναδική μέθοδο (σημείωση: η ανάθεση επίσης χρώματος (κίτρινο, κόκκινο, μπλε, μαύρο) στους παίκτες δεν υλοποιείται αφού δεν προσφέρει τίποτα). Επίσης, έχουμε τράβηγμα πλακιδίου ("drawTiles()"), επιλογή πλακιδίου και ενεργοποίηση κάρτα χαρακτήρα ("GetOneTileFromTheBoard()"), πόντοι ("CalculatePlayerPointsAtGameOver()"), σειρά παικτών ("SelectPlayer()"). Τέλος, ο έλεγχος νικητή συμβαίνει στον Controller όποτε πατάμε το κουμπί "DrawTilesButton".

[ΤΕΛΟΣ]